



Industrial Machine Learning on Hadoop and Spark

Maks Nakhodnov, Bremen 2025

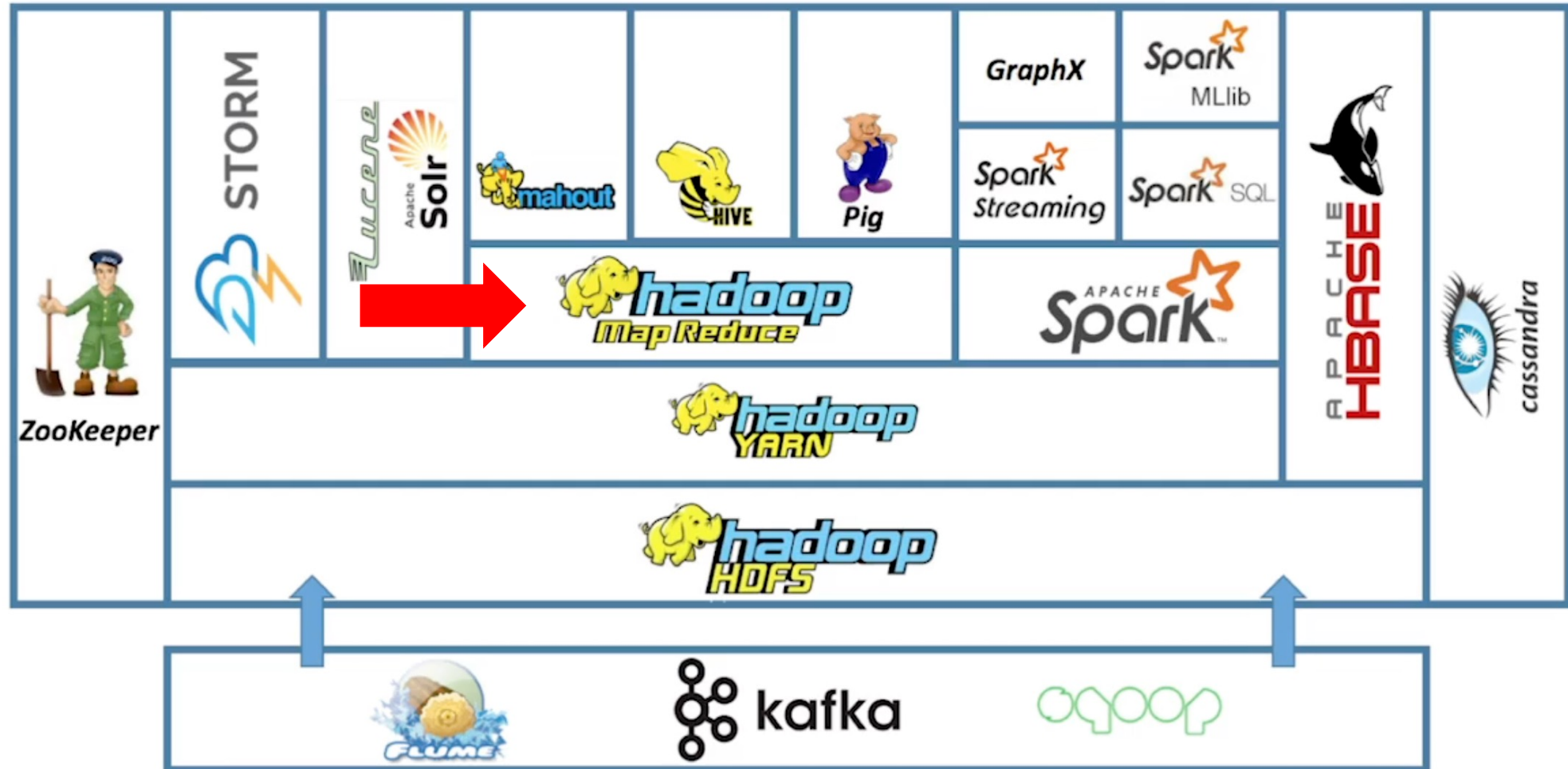
Distributed computing. MapReduce

Plan:

How to process big data?

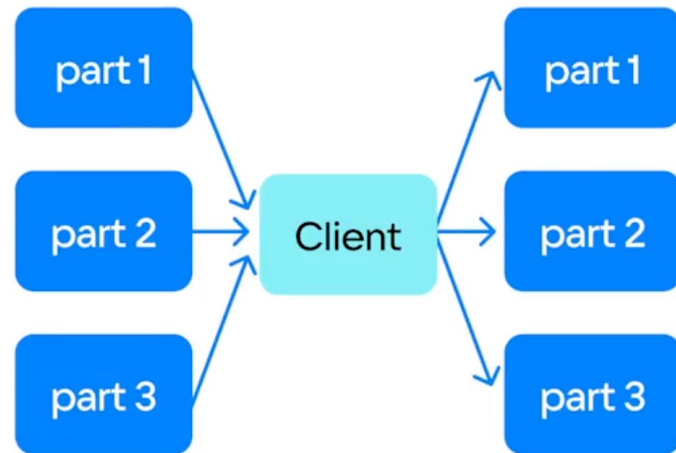
- MapReduce algorithm. Basic components.
- Hadoop MapReduce

Hadoop Ecosystem



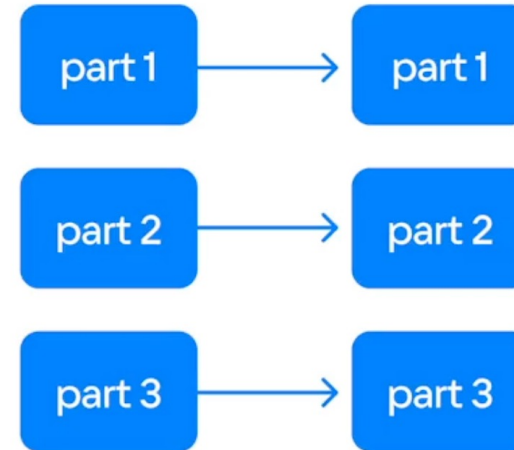
MapReduce. How to process data efficiently?

- Data locality
- Lack of efficient random access



hdfs dfs
-cp

MapReduce



hdfs distcp

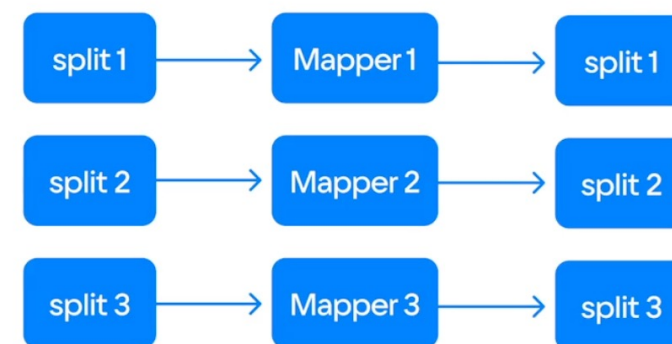
MapReduce. Massive data parallelism

- Most of the processing tasks can be separated into individual subtasks

I. Example: Remove punctuation (no reduction)

$$D = \{d_1, \dots, d_n\} \implies \hat{D} = \{\hat{d}_1, \dots, \hat{d}_n\}$$

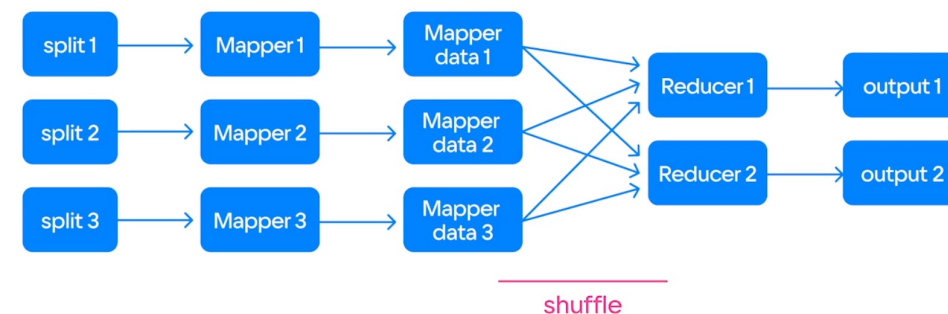
1. Process each document independently



II. Example: Build an index for the data corpus

$$D = \{d_1, \dots, d_n\} \xrightarrow{\text{to index}} \{w_i \rightarrow [d_{i_1}, \dots, d_{i_p}]\}$$

1. Process each document independently
2. Combine the results



MapReduce. Simple functional primitives

Stateless functional primitives that operate on immutable data:

- Allows reusability and **reruns**
- No shared memory
- Deterministic

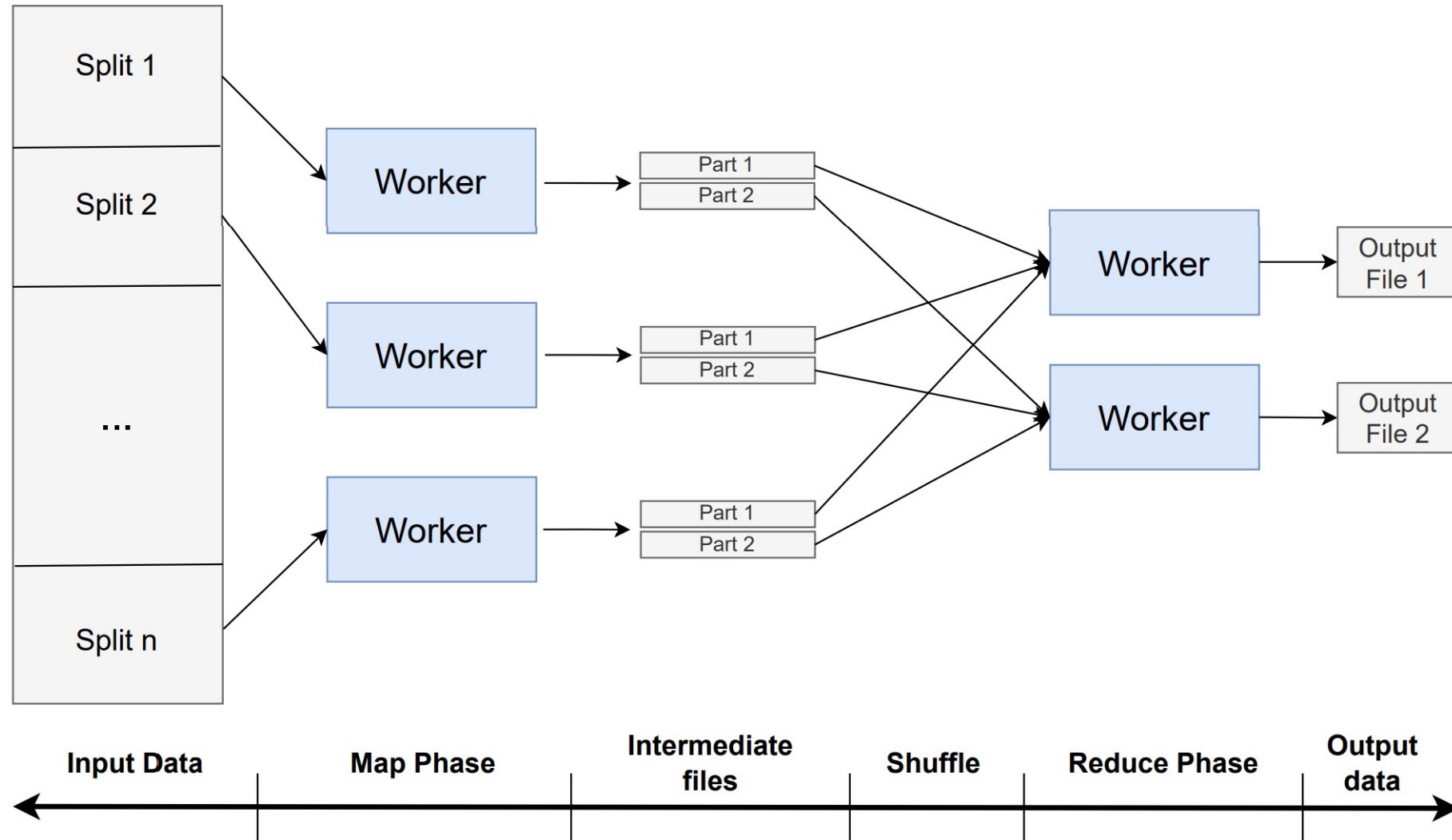
List one-to-one transformation:

$$\textit{map} : f, [v_1, \dots, v_n] \rightarrow [f(v_1), \dots, f(v_n)]$$

List reduction:

$$\textit{reduce} : g, [v_1, \dots, v_n] \rightarrow g(v_1, g([v_2, \dots, v_n]))$$

MapReduce Flow



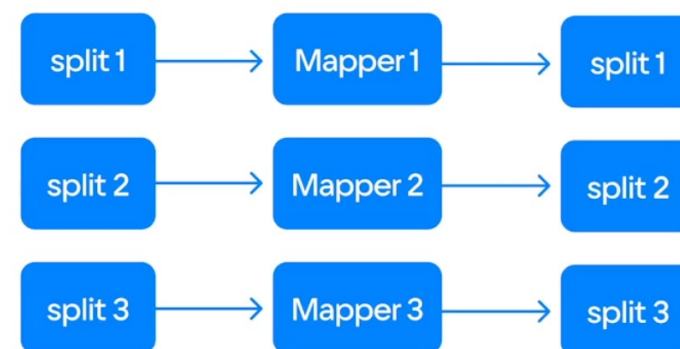
MapReduce Flow. Massive data parallelism

- Most of the processing tasks can be separated into individual subtasks

I. Example: Remove punctuation (no reduction)

$$D = \{d_1, \dots, d_n\} \implies \hat{D} = \{\hat{d}_1, \dots, \hat{d}_n\}$$

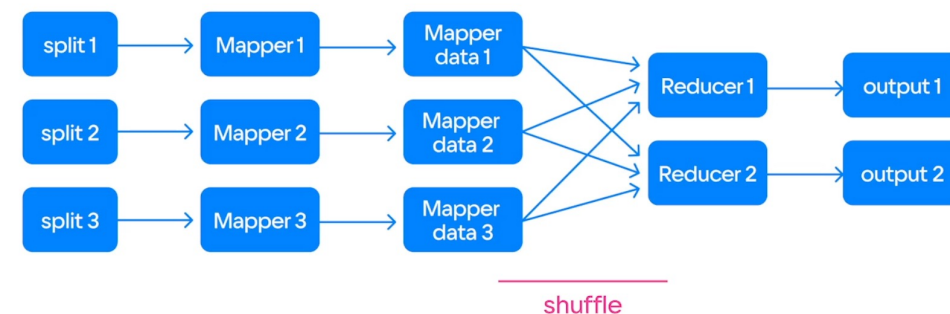
1. Process each document independently



II. Example: Build an index for the data corpus

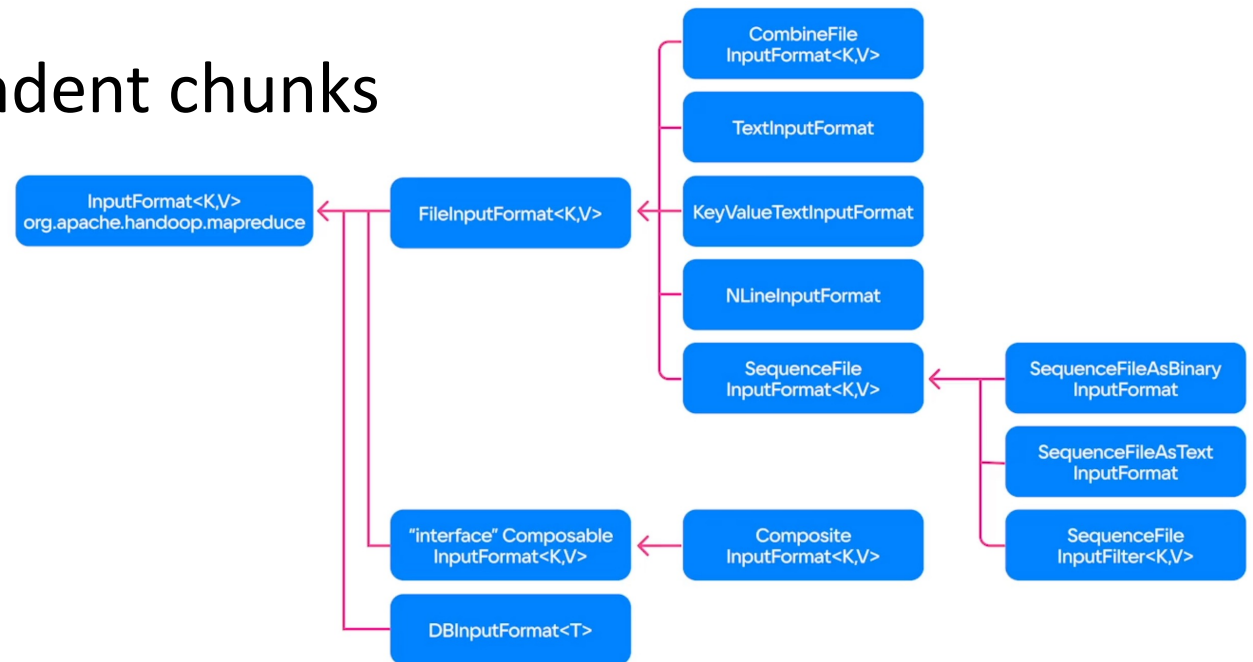
$$D = \{d_1, \dots, d_n\} \xrightarrow{\text{to index}} \{w_i \rightarrow [d_{i_1}, \dots, d_{i_p}]\}$$

1. Process each document independently
2. Combine the results



MapReduce Flow. Input Data

- A single file or a group of files
- Located in HDFS
- Allows it to be split into independent chunks
- A key-value representation
- Various data sources:
 - Various text formats
 - Splittable archives (.bzip2)
 - SQL databases (Cassandra)



MapReduce Flow. Input Data

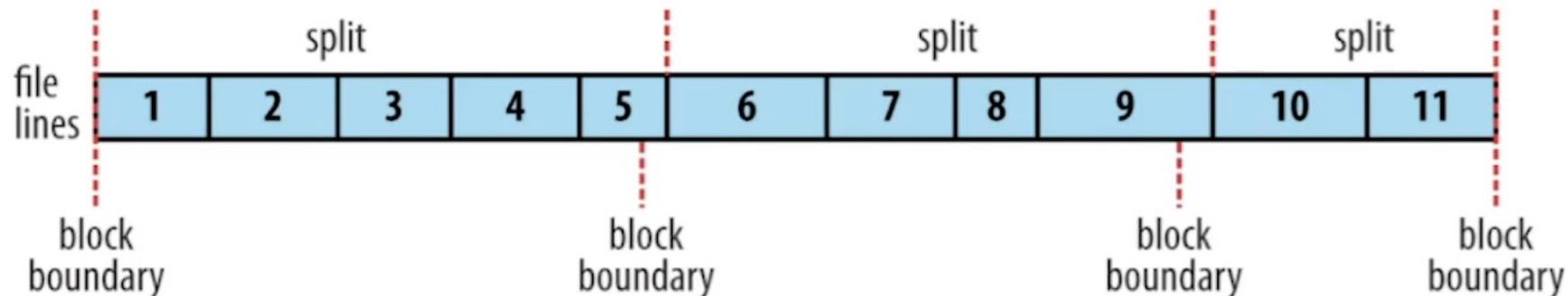
Split size: $\max(\text{minSize}, \min(\text{maxSize}, \text{blockSize}))$

maxSize $\rightarrow$ `mapred.max.split.size`

minSize $\rightarrow$ `mapred.min.split.size`

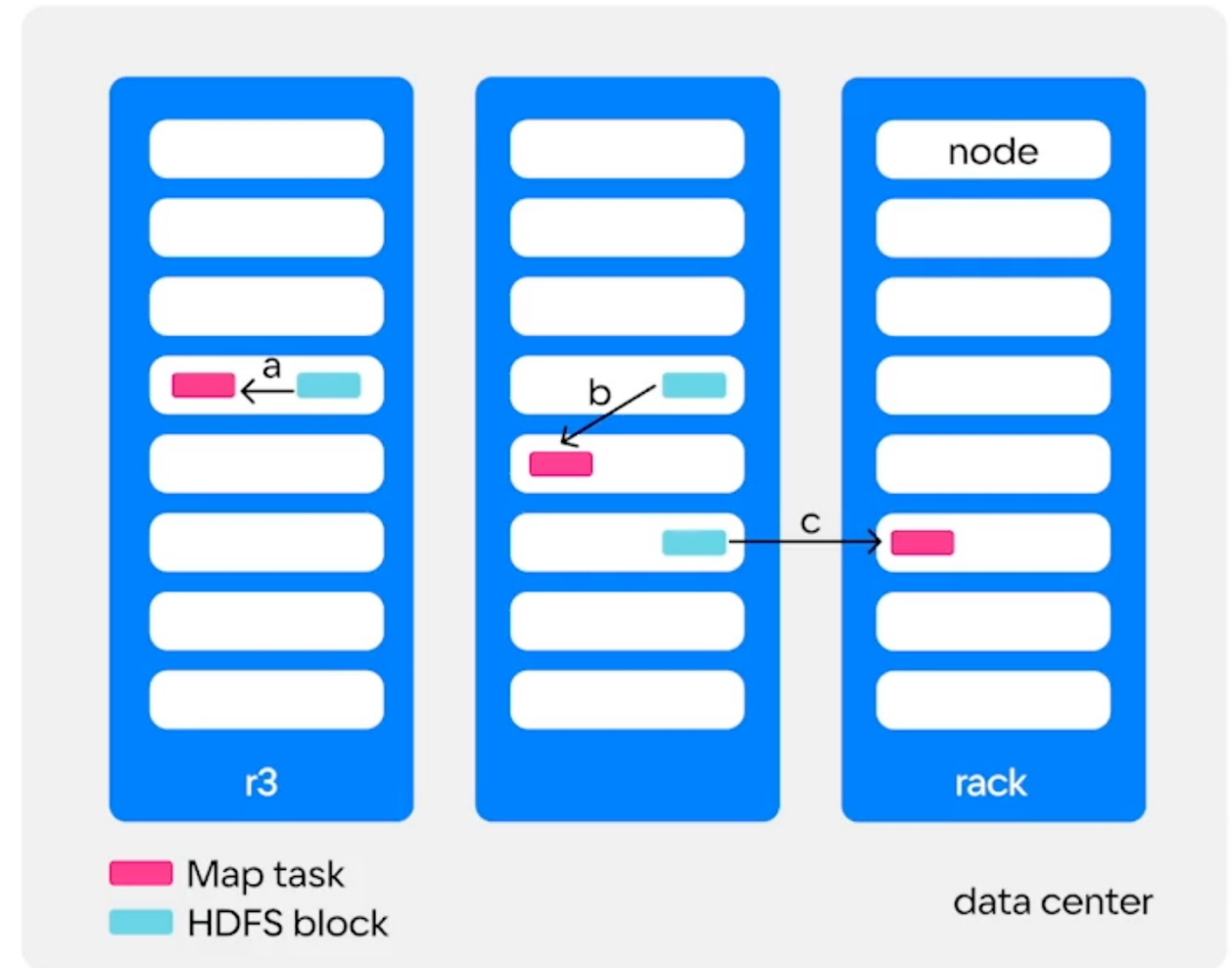
blockSize $\rightarrow$ HDFS block size

- Lower split size $\rightarrow$ better data locality
- Lower split size $\rightarrow$ better fault tolerance
- Larger split size $\rightarrow$ lower computational overhead



MapReduce Flow. Map Phase

- Independent workers
- Transforms the sequence of KV pairs from the split
- Number of mappers equal to the number of splits
- Tries to have data locality, but not as important nowadays



MapReduce Flow. Intermediate data

- Mapper output is partitioned by the key — *one* file for each reducer (Partitioner)
 - $P(K) = \text{hash}(K) \% \# \text{Reduces}$
 - *Spill files are created before the local merge* (multi-way merge sort)
 - Salting: key $\rightarrow$ (key, salt)
- Each file is sorted by key (SortComparator)
 - Distributed sorting
 - Allows for secondary sort
- Usually stored in the local FC (instead of HDFS)
 - Reduces replication overhead
 - Cause recomputations in case of failure

MapReduce Flow. Combiner

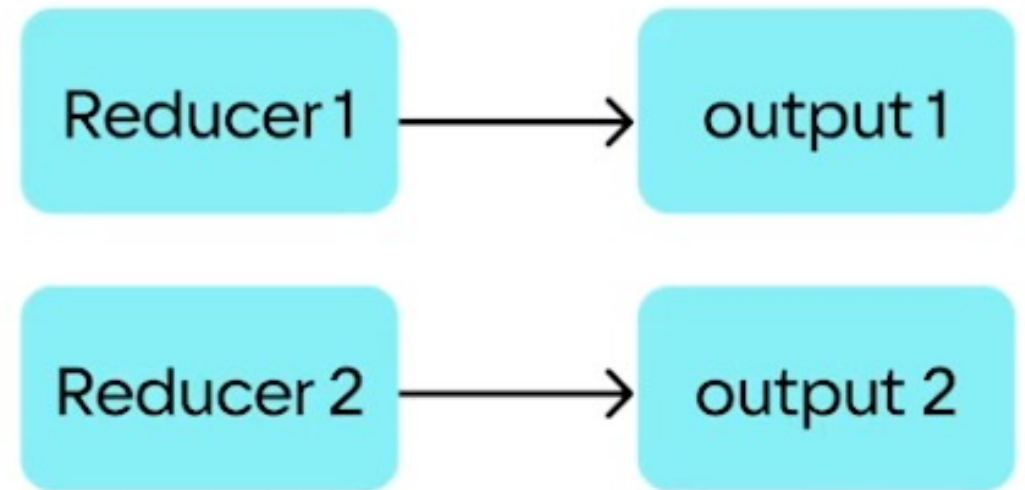
- The Optional Combiner Phase can perform reduction locally to decrease the amount of data transferred through the network
- Pure optimization execution: has to be an associative $(a \oplus b) \oplus c = a \oplus (b \oplus c)$ and commutative $a \oplus b = b \oplus a$ function
- Has to be compatible with Mapper output and Reducer input formats

MapReduce Flow. Shuffle

- The most network-heavy part of the framework
 - Key optimization factor
- Second part of the distributed sorting
 - Merge the sorted lists
 - Additional Combiner calls are possible
- The sorted list of keys is separated into groups of values with the “same” key (GroupingComparator)

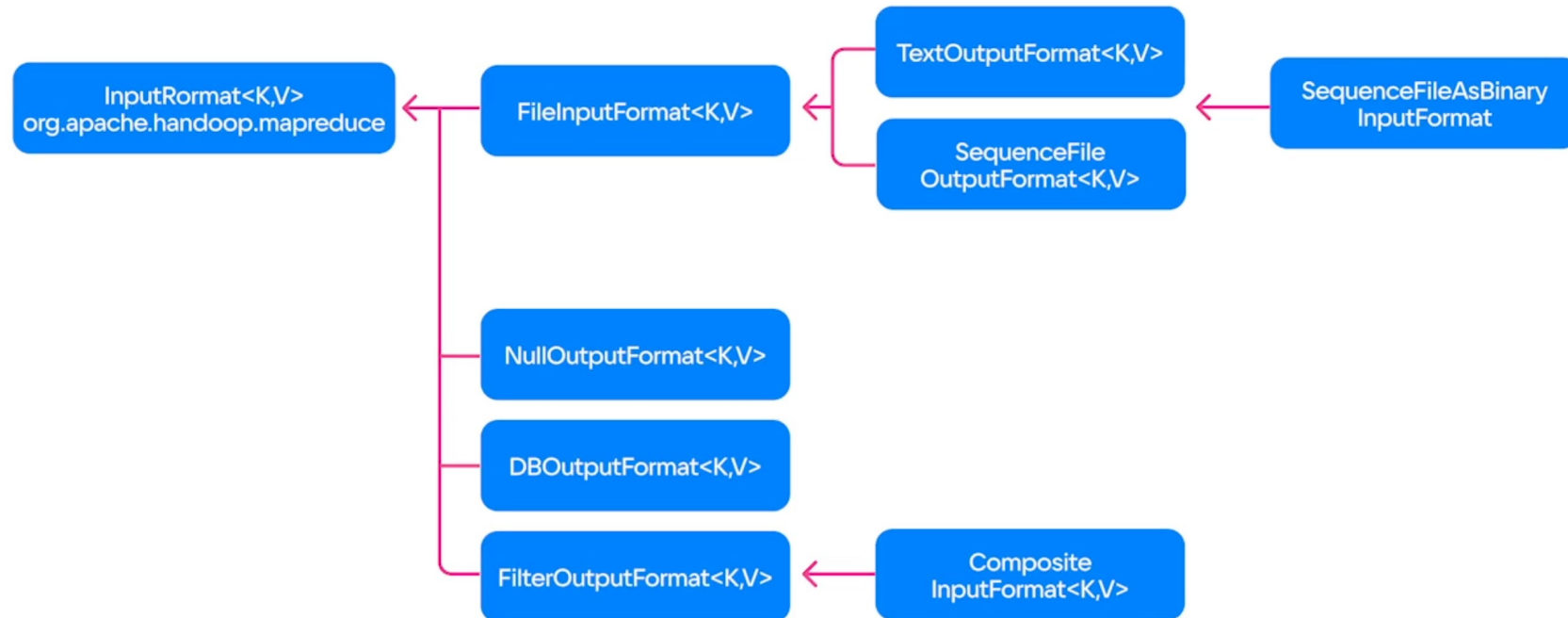
MapReduce Flow. Reduce Phase

- The sequence of groups (K, [V1, ..., Vp]) is passed to the reducer to be aggregated
- The resulting sequence of KV pairs is written into HDFS as it is



MapReduce Flow. Output Data

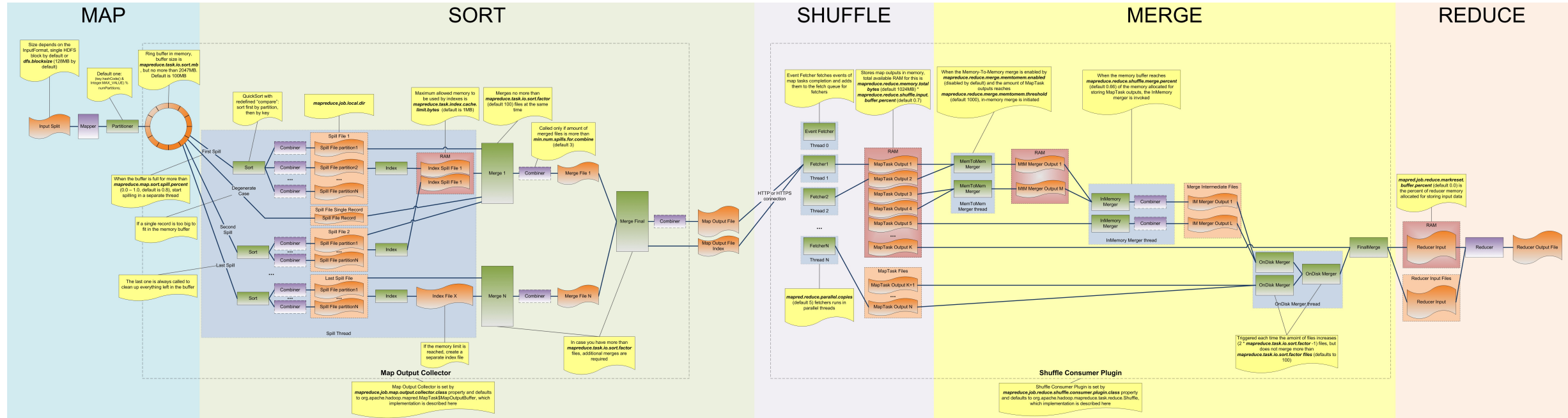
- Stored in HDFS
- The number of output files is equal to the number of reducers



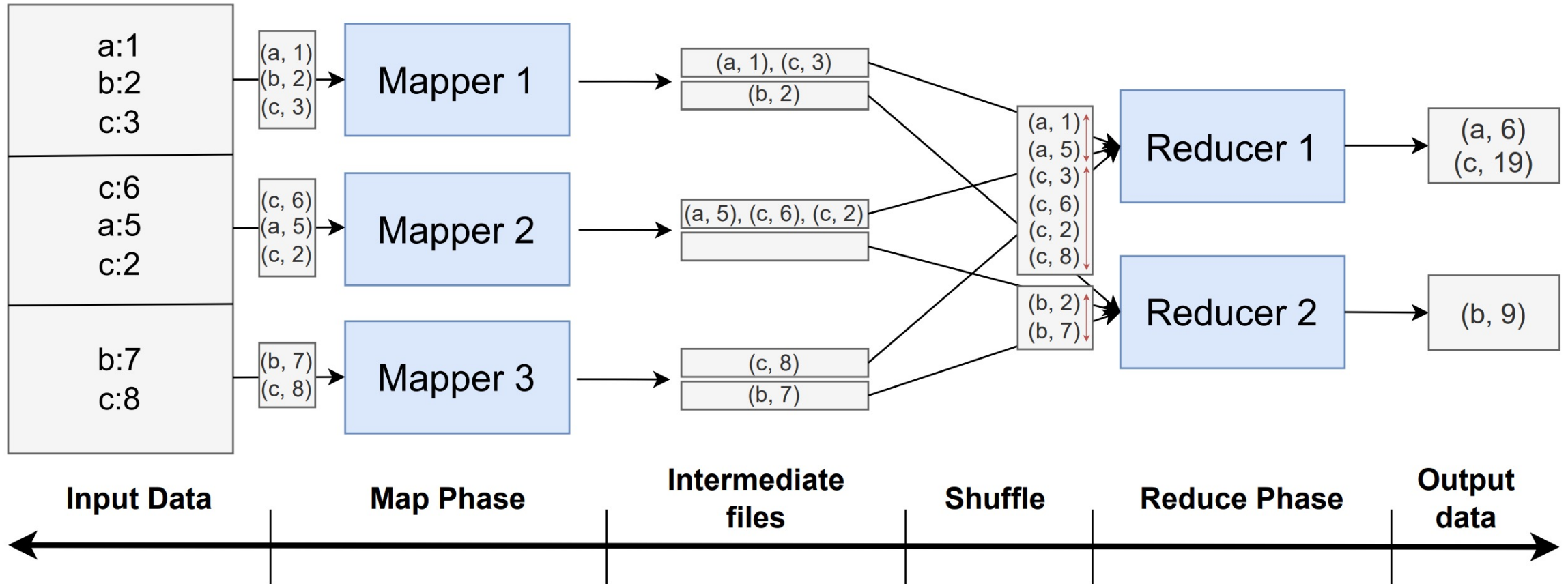
MapReduce Flow. What can you change?

1. Input data location
2. Input data KV representation and size of data splits
3. Mapper function
4. Partitioner
5. Primary Sort Comparator
6. Combiner
7. Secondary Sort Comparator (Grouping Comparator)
8. Reducer
9. Output data location

MapReduce Flow



MapReduce Flow. Example



MapReduce Flow. Fault tolerance

- Input and output data are stored in HDFS, which ensures their availability
- If any intermediate data is lost, just recompute it from scratch — **stateless, deterministic Map/Reduce is required**
- If the Map Phase takes too long, it might be beneficial to separate the MapReduce task into the Map-only and Reduce-only tasks to store intermediate results in HDFS

MapReduce Flow. Speculative execution

- Tasks (map/reduce) can be executed multiple times if the process stalls on one of the nodes
- The first result to be computed is accepted; others are discarded
- **Stateless, deterministic Map/Reduce is required**
- `mapreduce.map.speculative`
- `mapreduce.reduce.speculative`
- Helps against straggler-tasks and the “long tail”

MapReduce Flow. Reducer input imbalance

- The user has to ensure that all reducers are utilized equally
- The load imbalance can be solved using a custom partitioner

Example. Websites Processing: (URL, Content) —> Statistics

- URL can be distributed unequally
- i.e., google.com is much more frequent
- Naïve hash partitioning will create load imbalance

MapReduce. Counters

- Mechanism for monitoring and collecting non-important results
- Expensive to update
- Can only be a 64-bit integer
- Non-deterministic by design (i.e., task reruns)
- Available from UI to monitor execution
- Either built-in or user-defined:

